Co-funded by the European Union

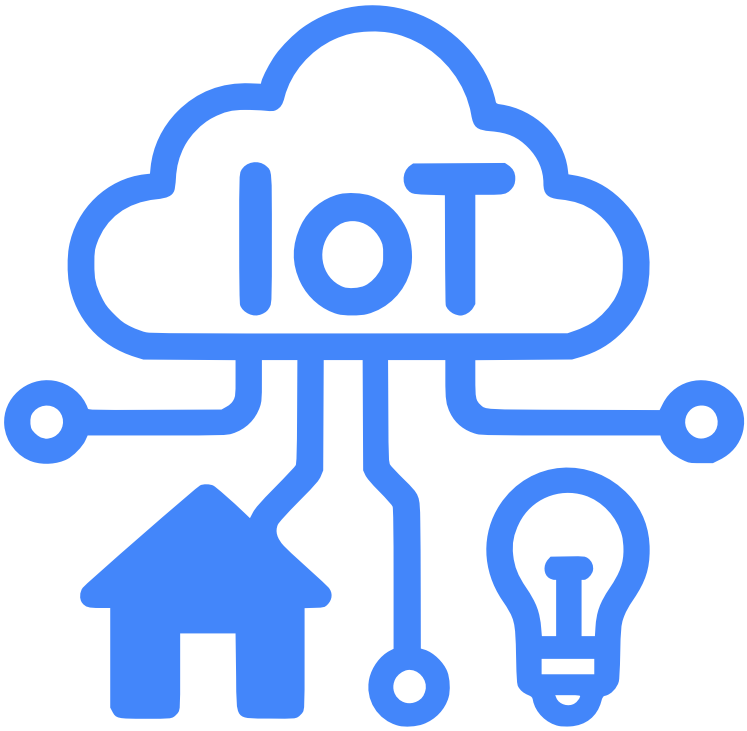
Gerda Stetter Stiftung
Technik *macht* Spaß!

EDUDEMOS

EDUcating through Sustainable **DEMO**nstrators

A guide to

Internet of Things



Gerda Stetter Stiftung Technik *macht* Spaß! Finnova gbs sg.ch FUNDACIÓN SERGIO ALONSO I.E.S. El Rincón

V1.0

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Education and Culture Executive Agency (EACEA). Neither the European Union nor EACEA can be held responsible for them.

EduDemoS © 2025 by Gerda Stetter Stiftung – Technik macht Spaß, Fundación Sergio Alonso, FINNOVAREGIO, GBS St. Gallen and IES El Rincón is licensed under

Creative Commons Attribution-NonCommercial 4.0 International

To view a copy of this license, visit https://creativecommons.org/licenses/by-nc/4.0/

Table of contents

1 About this guide 4

2 A brief introduction to the Internet of Things 5

3 Connecting your demonstrator 7

4 Visualizing your demonstrator 8

4.1 Figuring out what telemetry exists 8

4.2 Finding your dashboard 10

4.3 Visualizing solar values (scalar values) 11

4.4 Visualizing LDR sensor values (JSON-objects) 13

5 Controlling your demonstrator 15

5.1 Visualizing the mirror status 15

5.2 Controlling the mirror status 15

6 And now? 19

7 Some background information 20

7.1 About machine-to-machine communication 20

7.1.1 A mental model for communication 20

7.1.2 The MQTT protocol 20

7.2 About ThingsBoard 23

7.2.1 Dashboards 23

7.2.2 Devices 23

7.2.3 Telemetry 24

7.2.4 Remote Procedure Calls (RPC) 24

7.3 How the software solution works 24

8 Setting up your own workshop 25

1 About this guide

Note: For following this guide, it's necessary to have built one of the demonstrators which are IoT ready. The demonstrators are tagged with "IoT-ready" on their README.md-page in their respective EduDemoS repository.

This guide provides you with information to turn your demonstrator into an IoT device. If you are new to the topic of Internet of Things, chapter 2 offers a very brief introduction into the matter. The actual hands-on chapters are chapter 3, giving you guidance on how to prepare the demonstrator for IoT readiness, and chapter 4, instructing you on how to visualize information sent by the demonstrator.

While chapter 5 gives you an insight into the slightly more advanced topic on how to send information from the dashboard to your demonstrator, chapter 6 offers a couple of ideas on what to do next with the newly learnt abilities. And if you are keen on diving deeper into the details of the technologies, chapter 7 provides you with some extra information and bootstraps for further investigation.

Great you are taking on the topic!
Have fun creating your own small IoT system!

2 A brief introduction to the Internet of Things

Note: This section gives you a brief overview of the system you are going to work with when connecting your demonstrator. It's ok to skip it if you already heard about the Internet of Things from your workshop host or elsewhere or if you are eager to get to action.

However, it's worth getting back here if you run into any problems during your experiments.

You might have heard people talking about the "Internet of Things" or "IoT" for short. But what does that actually mean? – Simply put: IoT is all about machines and gadgets talking to each other over networks like the internet – without human intervention.

For most people, IoT is closely connected to smart homes such as switching lights with your phone or even automatically based on your current location or having a fridge ordering milk on its own. However, IoT is way more than that! Imagine a trash bin that knows when it's full and automatically calls a worker to empty it. Or sensors counting and reporting people walking through dangerous areas, so safety teams know where to look for missing people. But also, large industrial machines can be "smart" by ordering their own repairs before they break down.

In short: IoT connects all sorts of devices with the goal of making life easier, safer and sometimes even more fun by letting machines talk to each other.

But how does it work? One of the most common system topologies is depicted in Figure 1, based on the EduDemoS sun-tracker demonstrator. While machines could talk to each other directly, the systems are often simplified by adding a central information broker to the system. The broker makes sure that the information can always be sent by devices ("published by the producers") providing new data and the information can always be queried by interested services ("consumed by subscribers").

But why is it done that way? – Not all devices are always online. In fact, maintaining a communication link is quite power consuming (especially when it comes to wireless communication). Therefor battery powered devices tend to save power by shutting down and only wake up occasionally to see if there is any new information to collect and update their status. By having a broker which is always up and running, all devices and services can connect at any time for publishing / consuming data of interest. Even if the device is not connected for the moment.

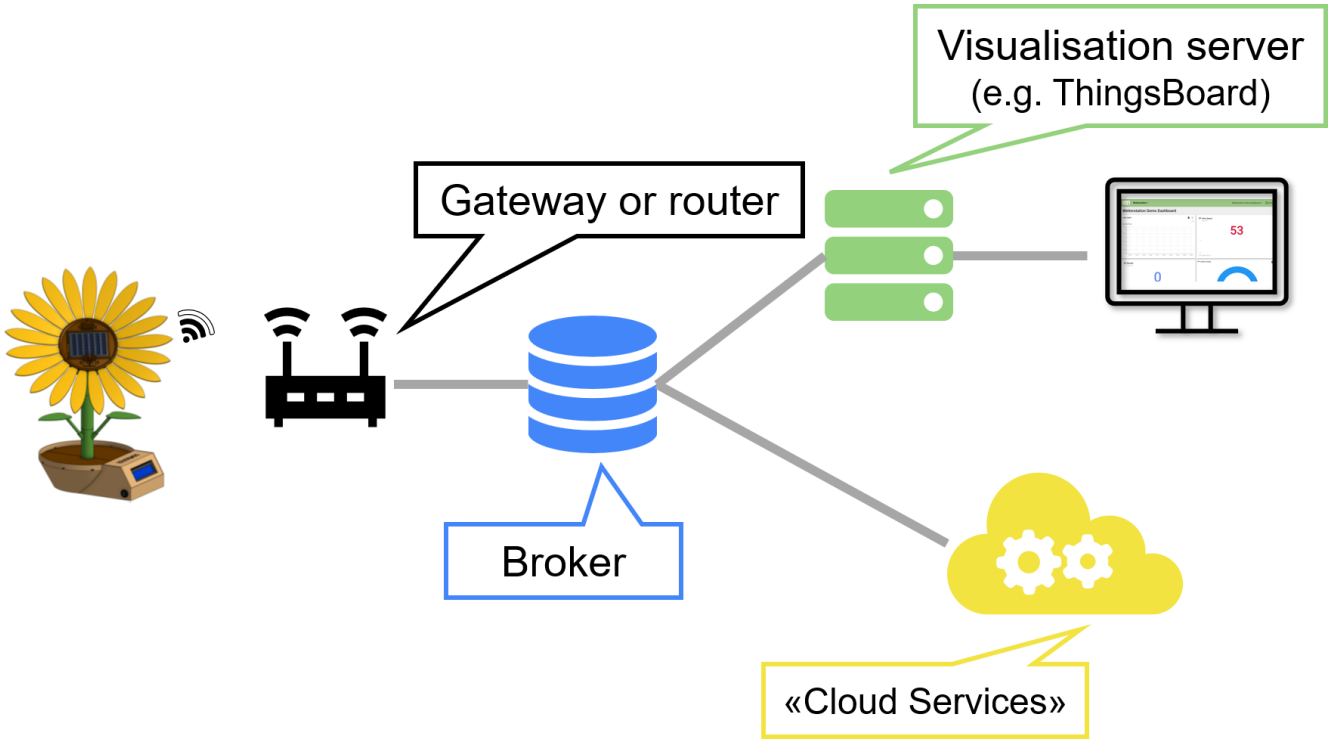


Figure 1: An example of a small IoT system

Based on the data published by any device, various services can take decisions and maybe even calculate new instructions to be sent back to the publishing device or other devices depending on the information for decision making. Other services may simply archive the live data for later use or collect the data to render a cockpit or dashboard giving an operator a quick overview of the status of the whole system.

3 Connecting your demonstrator

In order to enable your demonstrator for IoT applications, software capable of sending data to a broker is necessary. This software can be downloaded from the demonstrator's repository on GitHub:

https://github.com/EduDemoS/

> **Caution:** Make sure you pick the correct software for your demonstrator. Failing to do so might cause hardware damage.

The following instructions are valid for any of the IoT-ready demonstrators.

1. Navigate to the repository of your demonstrator.
2. On the `README.md` page, locate the link to the directory containing the IoT-ready software and click it.
3. Download the files in the directory (it's not necessary to download `README.md` and `.gitignore`).
4. Follow the instructions in the `README.md` of that folder to adapt the project for your local situation. Ask your workshop host if you miss any information or credential information.
5. Compile and upload the software to your demonstrator.

> **Warning:** Publishing your local copy of the software project risks exposing secrets such as SSIDs, usernames, passwords, etc.
>
> Hence make sure you strip any sensitive information from the project before sharing!

4 Visualizing your demonstrator

The **following instructions assume you logged on to your ThingsBoard instance** using credentials provided by your workshop host. It's also expected that your demonstrator is up and running as described in chapter 3.

The instructions take you through the steps necessary to visualize various information types in various ways.

4.1 Figuring out what telemetry exists

As a first step it's necessary to figure out what data, so called "telemetry", is offered by the demonstrator you try to visualize. In order to get an answer to this question, the following steps are necessary:

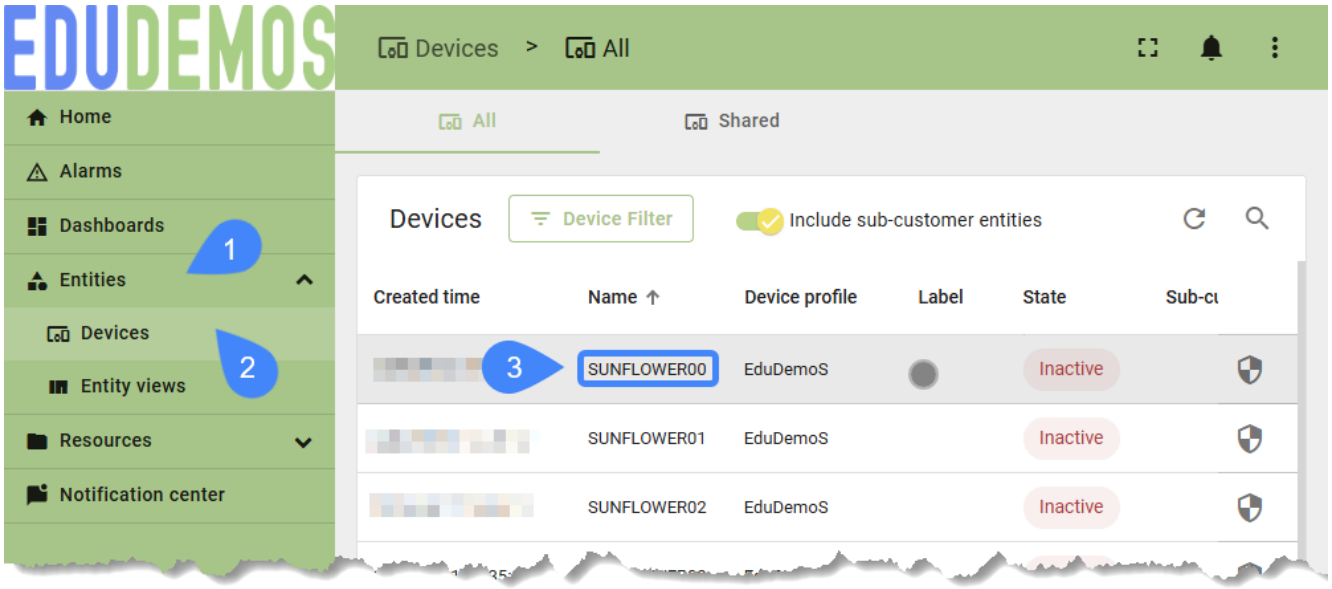


Figure 2: Navigating to your device in ThingsBoard

1. In the navigation on the left, open the group named `Entities`.
2. From the group, select `Devices`.
3. Select the device matching the demonstrator type and team number assigned to you by your workshop host.
4. After selecting the device, a window pops up. In that window, select the `Last Telemetry` tab (see Figure 3).

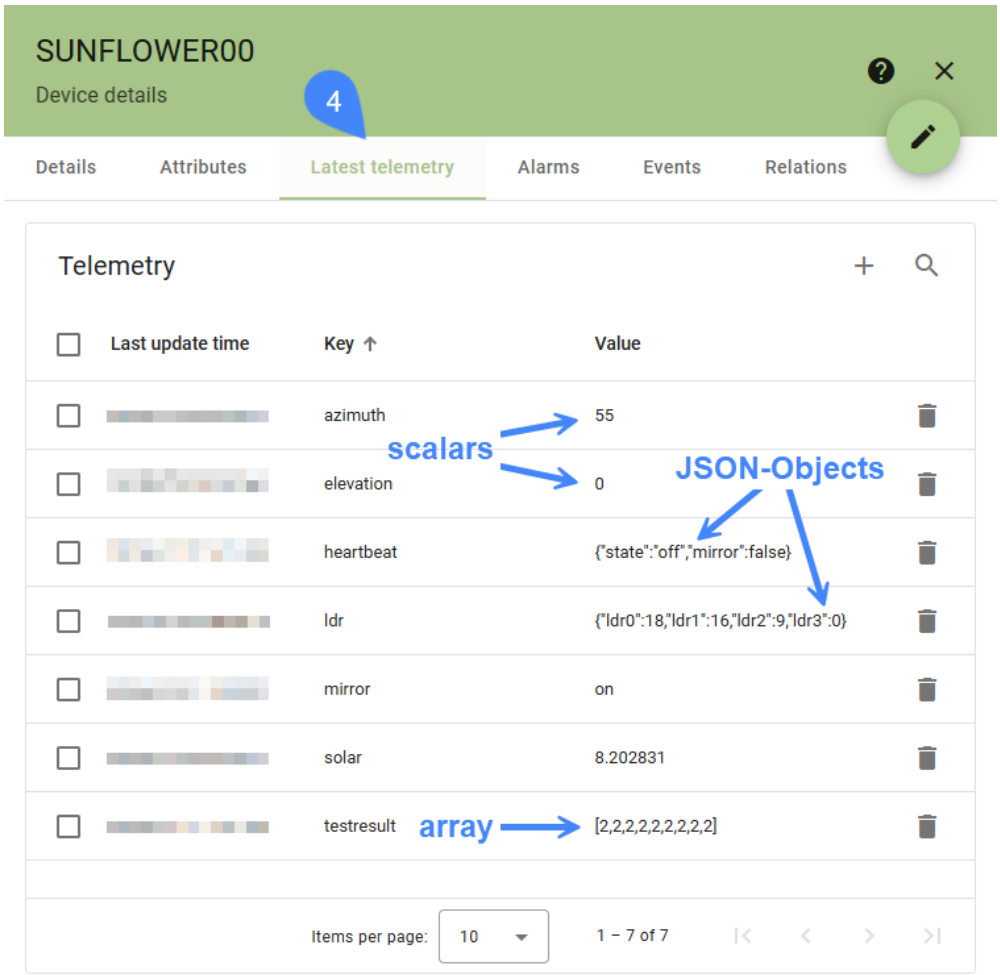


Figure 3: Overview of available telemetry for a device

This gives you an overview of available telemetry as shown in Figure 3. The telemetry is arranged in so called key-value pairs[1]. The values for the telemetry can be of various structures:

- Scalar values (e.g. `azimuth`, `elevation`, `mirror`, `solar`)
  (values not embedded within curly brackets)
- Arrays (e.g. `testresult`)
  (comma separated list of values,
- JSON-objects[2] (`heartbeat`, `ldr`)
  (key-value pairs, embedded in curly brackets)

In order to visualize the data, search for the information you want to visualize and note down the key and the value as an example for later reference. While visualizing scalars

[1] Key-Value pairs: Each information is assigned a unique name that can be used to access the data behind.
[2] JSON: A text based data structure, based on key-value pairs, allowing to store complex data structures. (see http://en.wikipedia.org/wiki/JSON or http://www.json.org for in depth information)

is quite straightforward (see chapter 4.2), visualizing JSON-objects as well as arrays demand extra steps (see chapter 4.4).

4.2 Finding your dashboard

To visualize the data, a so-called dashboard (see chapter 7.2.1) has to be configured. For your workshop, a dashboard for each team has been created in advance. You can find it by navigating as follows:

1. In the navigation on the left, select `Dashboards`.
   → A list of existing dashboards is displayed.
2. In the list, search the dashboard, named after your team (as provided by your workshop host).
3. The click opens your dashboard which is possibly still empty.

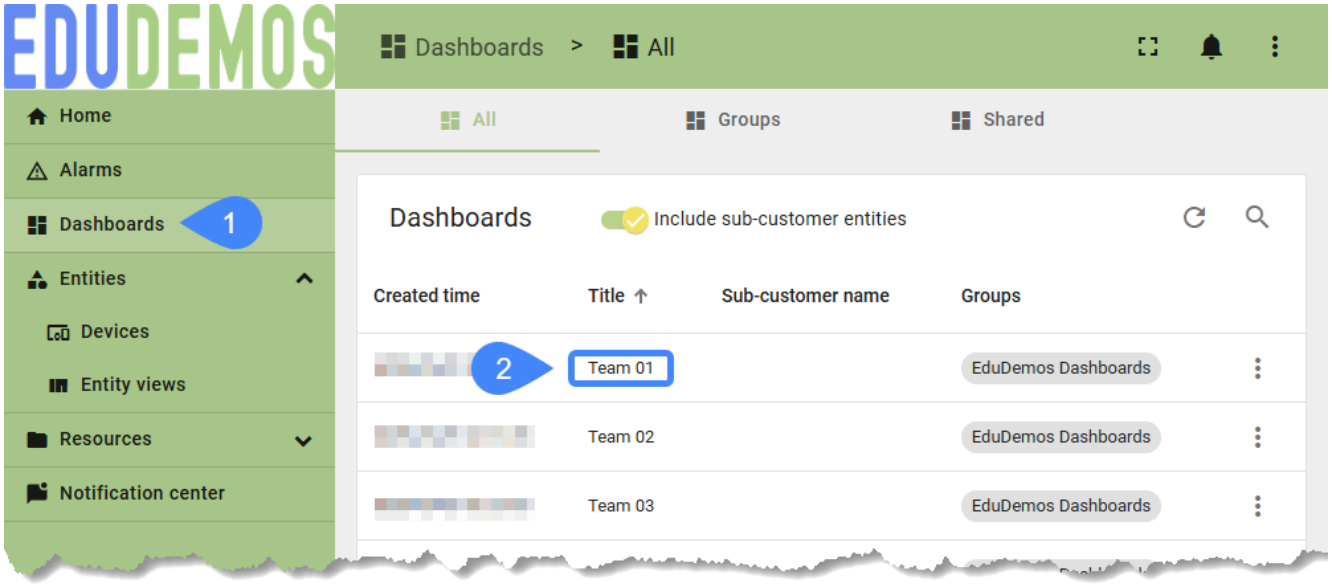


Figure 4: Navigating to your dashboard

4. If you intend to modify your dashboard, make sure the dashboard is in editing. This can be told from the toolbar on top of the dashboard as shown in Figure 5.
5. (optional) If you want to activate the editing mode, simply click the pencil icon (see Figure 5) in the toolbar.

Viewing mode

Editing mode

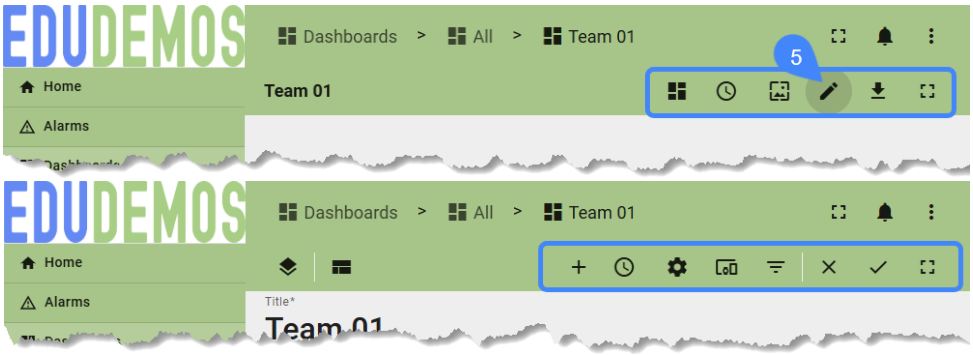
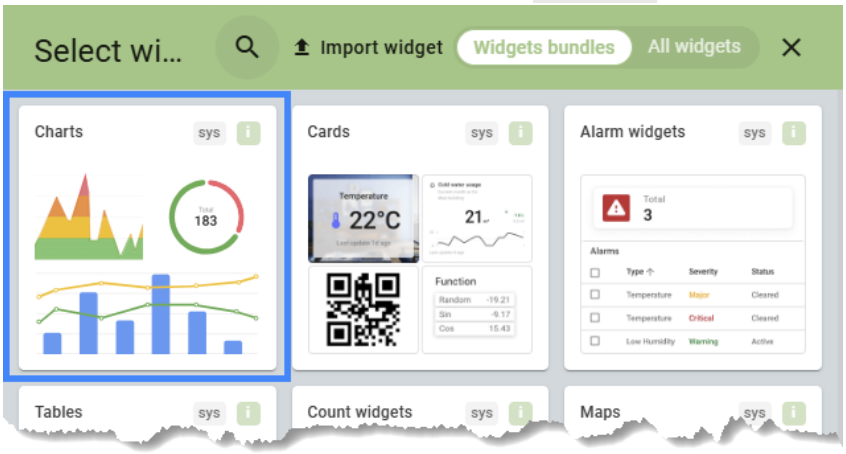


Figure 5: Distinguishing dashboard viewing- and editing mode

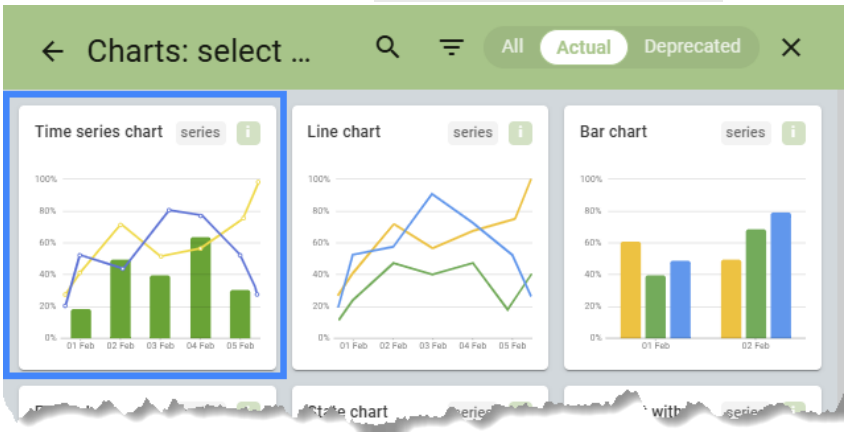
4.3 Visualizing solar values (scalar values)

As a first experiment, we try to visualize a scalar value in form of a line chart. For this example, we use the reported voltage of the solar panel (key `solar`). Of course, these instructions can be applied to any scalar information in the telemetry.

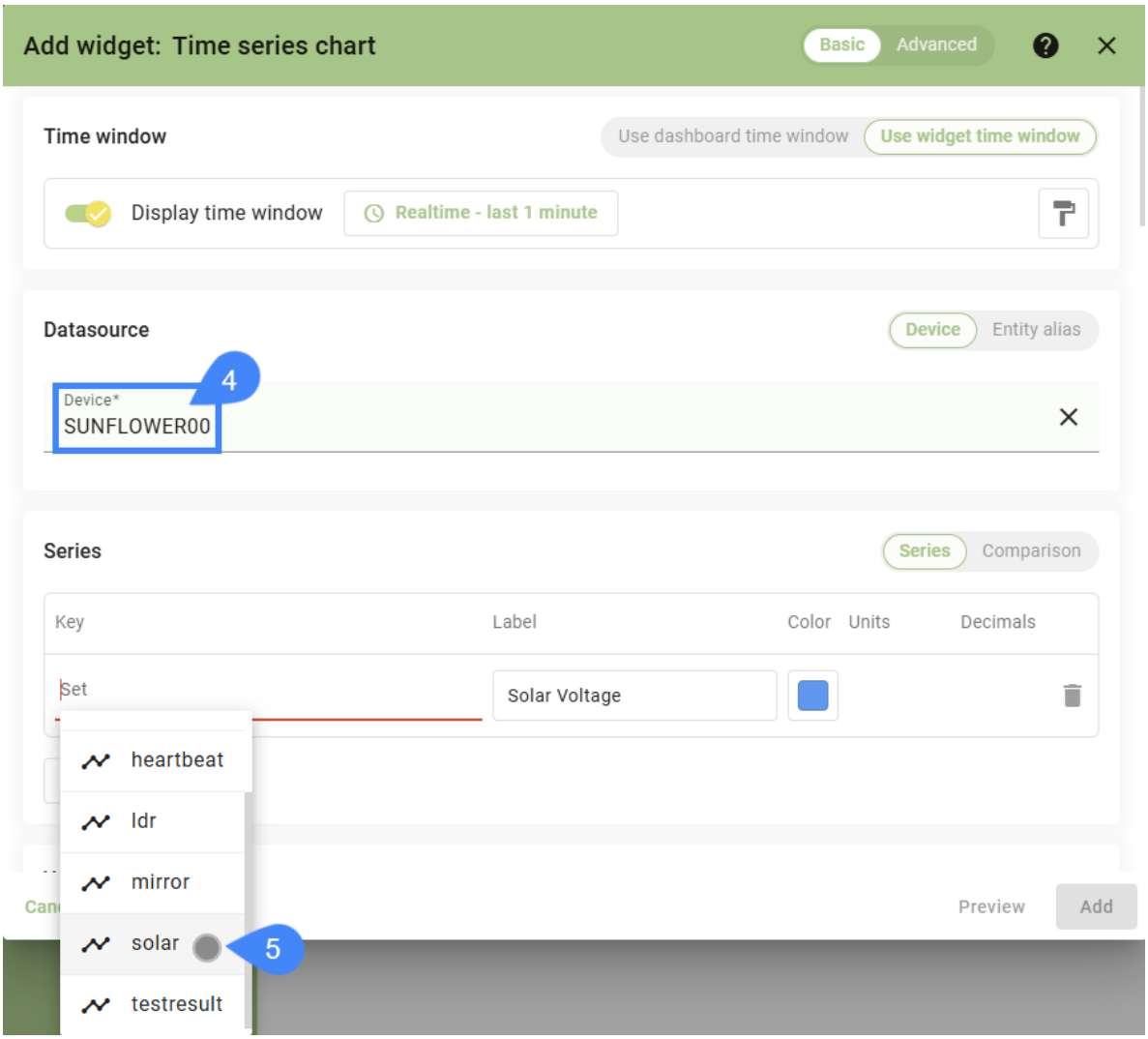
6. Make sure your dashboard is opened and in editing mode (see chapter 4.2).
7. Add a new widget by clicking the plus-icon in the toolbar.
8. From the widget gallery, select the `Charts` category.



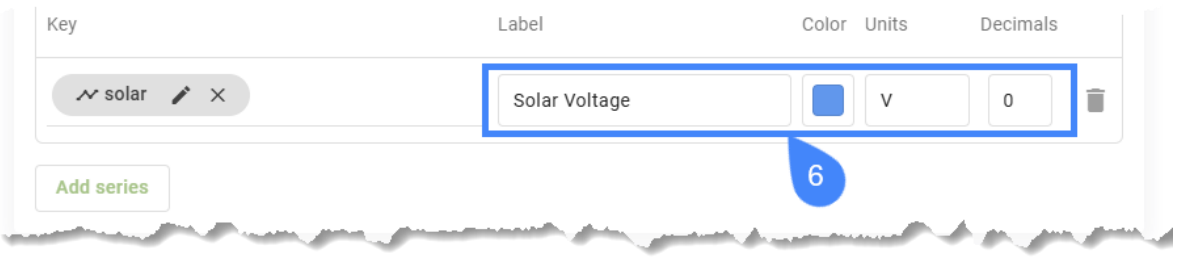
9. In that category, choose `Time series chart`:



10. Choose your device based on the name assigned to you by your workshop host.



11. Select the appropriate key (e.g. `solar`) for the data to visualize.
12. Setup `Label`, `Unit` and `decimal digits`.



13. Scroll to `Appearance` and set an appropriate diagram title.
14. Hit `Add` to add the widget to your dashboard.
15. If you like you can now grab the widget, relocate or resize it as you wish.
16. To test your dashboard, hit `Save` (checkmark icon).

4.4 Visualizing LDR sensor values (JSON-objects)

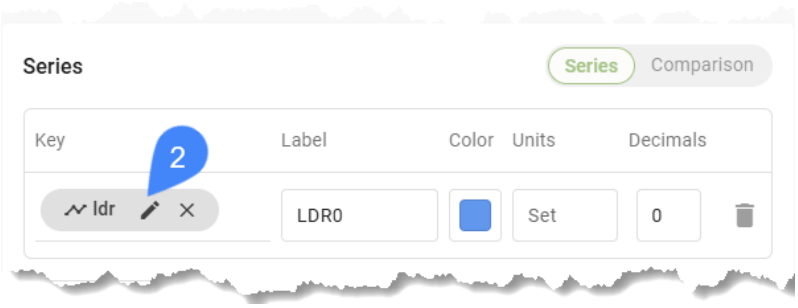
If information must be consistent[3], data is typically sent in complex objects, so a complete system status is updated at once. The downside of such an approach is, that complexity for visualizing that data increases slightly.

Looking at the JSON-object representing the LDR sensory, we get the following object structure:

```
{
  "ldr0": 30,
  "ldr1": 24,
  "ldr2": 0,
  "ldr3": 31
}
```

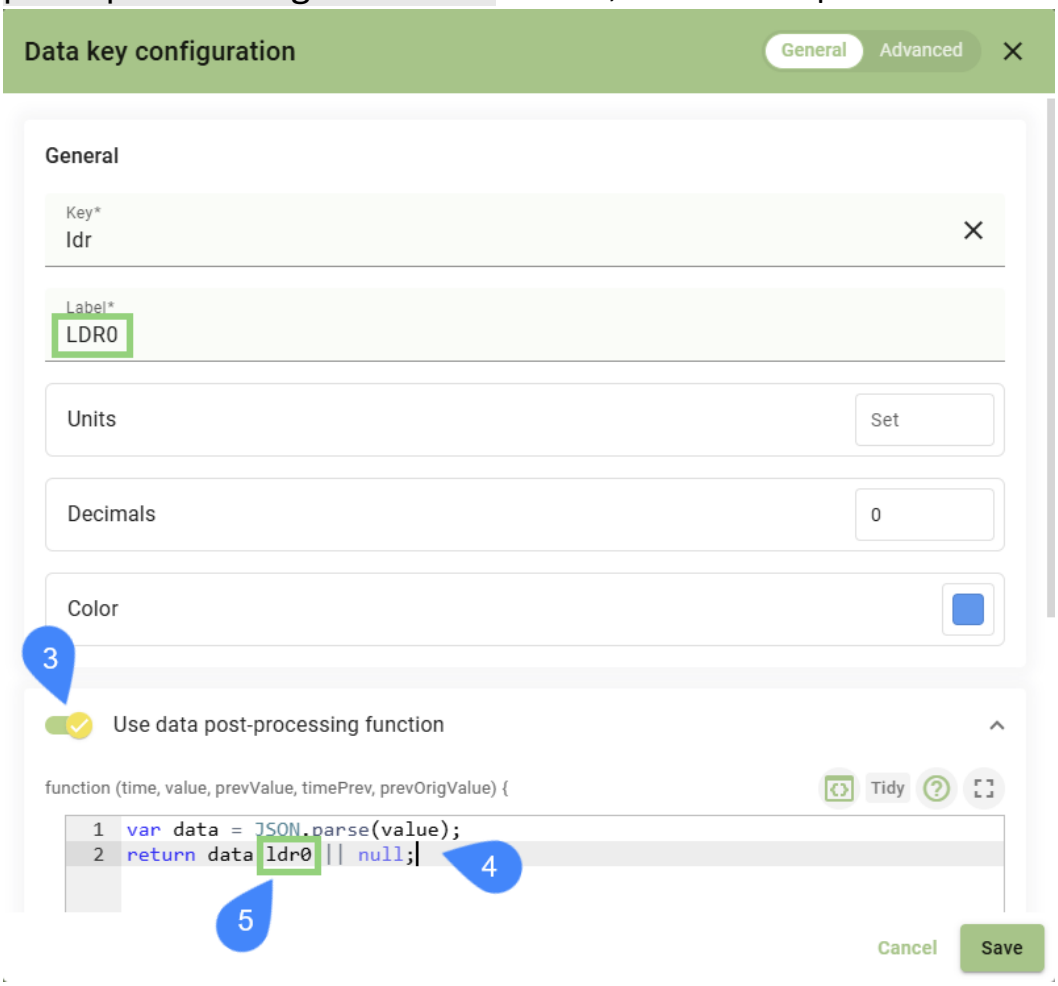
As we can guess, each LDR is represented by one of the keys. As we can see, the numbering starts at 0 instead of 1, as it is very common in software engineering. In order to access any of the keys within that JSON-object, we need to setup the datapoint slightly differently as described below:

0. Make sure your dashboard is opened and in editing mode (see chapter 4.2).
1. Follow instructions 7 through 12 in chapter 4.3 to add a time series chart and add `ldr` as series.
2. Clicking the pencil-icon for the data series opens the data series configuration dialog.



[3] Consistency: Sometimes, especially in IoT environments, it's sometimes important, to have multiple datapoints transferred at the same time to avoid contradicting status information.

3. For extracting the value for `ldr0`, we need to additionally configure a `data post-processing function`. Hence, switch this option on.



4. As code for the function, place the following snippet into the text field underneath the switch:

```
var data = JSON.parse(value);
return data.ldr0 || null;
```

5. Replace `ldr0` with the appropriate key for the datapoint you want to extract from the object.
6. Execute instructions 13 through 16 chapter 4.3 to add the widget to your dashboard, adjust it to your needs, and save the dashboard.

Very good!

You are now able to visualize various types of information on your dashboard.

5 Controlling your demonstrator

The **instructions assume you logged on to your ThingsBoard instance** using credentials provided by your workshop host. It's also expected that your demonstrator is up and running as described in chapter 3. The software implements not only an uplink sending status information upstream towards the dashboard, it also features a datapoint taking commands from the dashboard (downlink).

The functionality is simple: There is a telemetry datapoint named `data/mirror`. This datapoint contains either the value `on` or `off`. Whenever a method named `ctrl/mirror` is invoked, the datapoint is changed to the inverse of the argument of the method. This mechanism is illustrated in Figure 6.

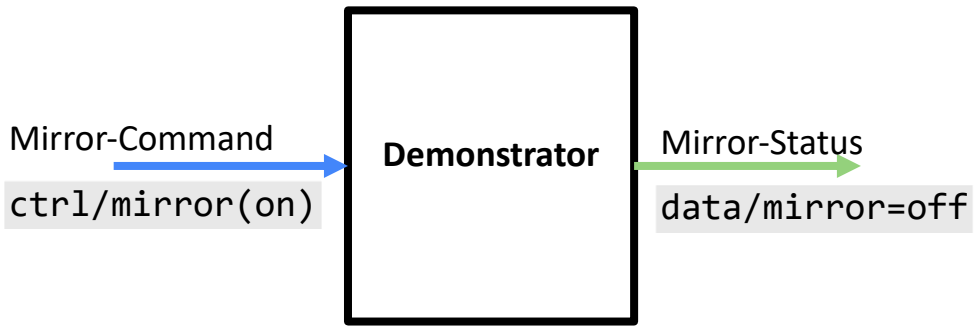


Figure 6: Block diagram for the mirror feature

5.1 Visualizing the mirror status

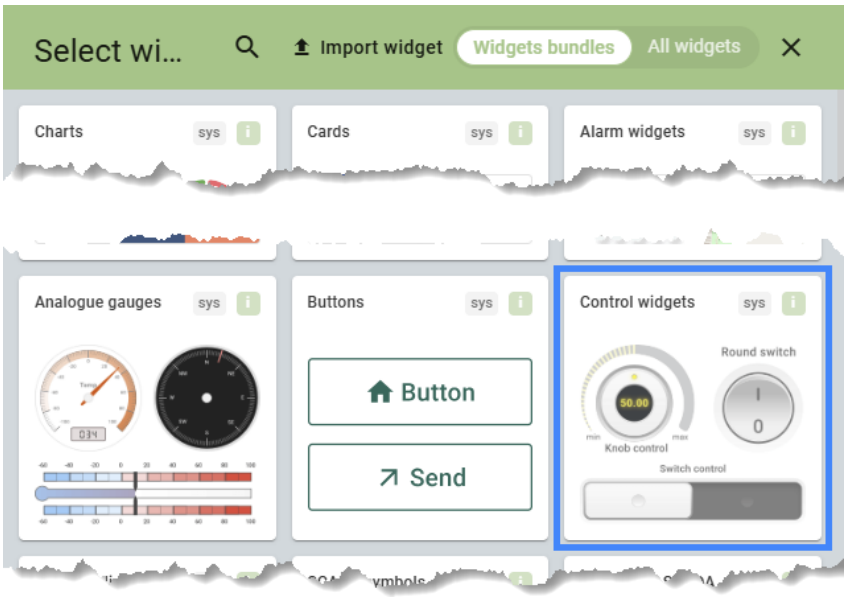
The first thing to do is to visualize the status for `data/mirror`. Follow the instructions outlined in chapter 4.3. However, instead of a `chart`, add `card`, namely a `value card` for the telemetry datapoint named `mirror`.

5.2 Controlling the mirror status

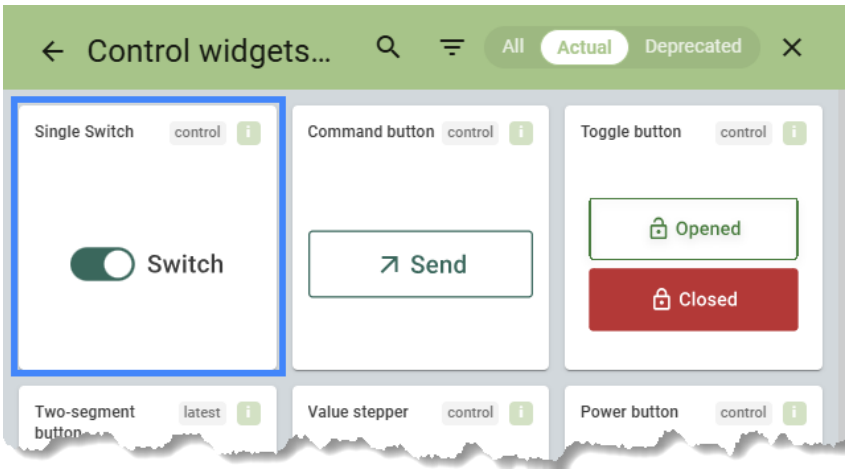
In order to control the mirror-status output, a control widget has to be placed on your dashboard.

17. Make sure your dashboard is opened and in editing mode (see chapter 4.2).
18. Add a new widget by clicking the plus-icon in the toolbar.

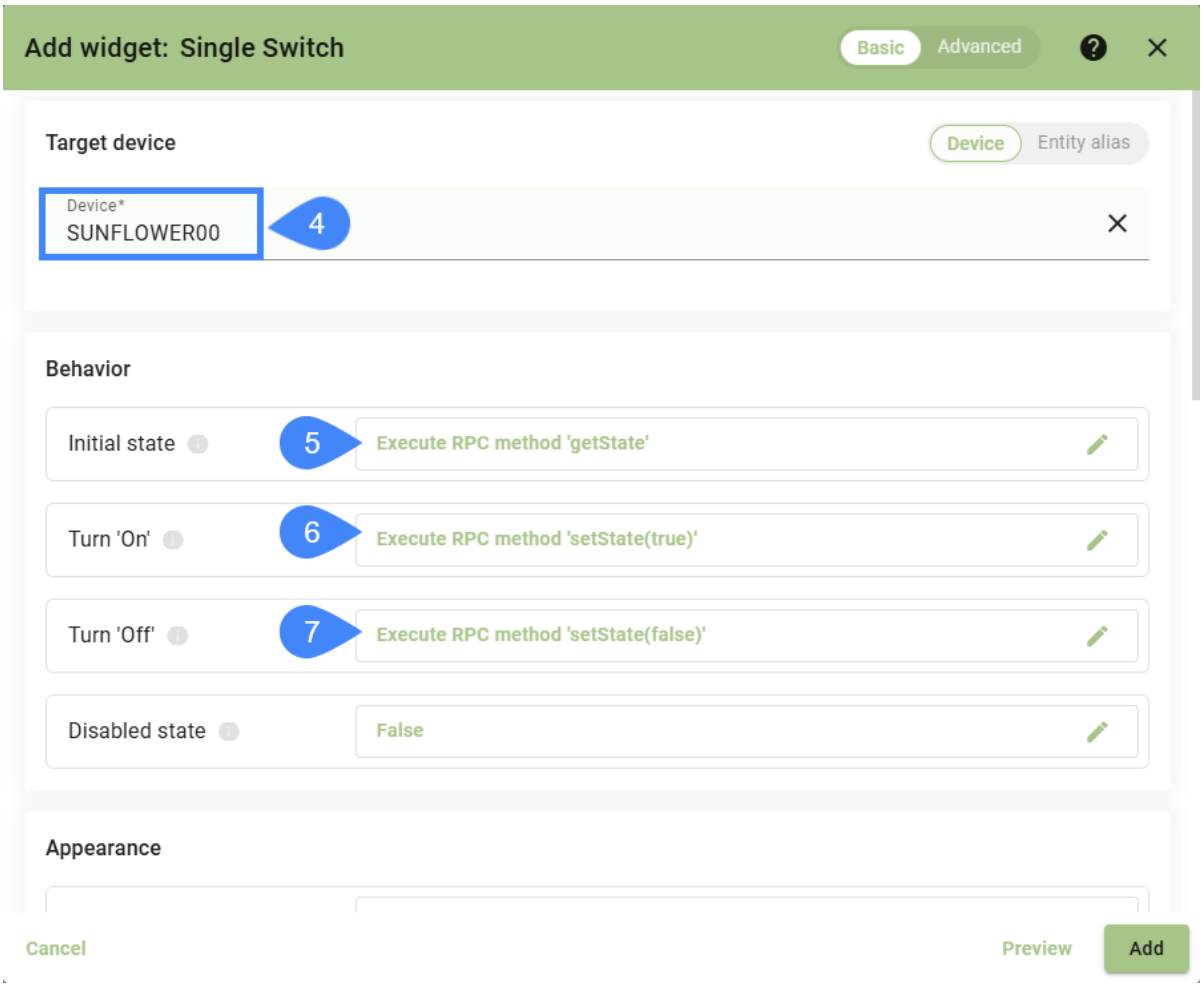
19.From the widget gallery, select the Control widgets category.



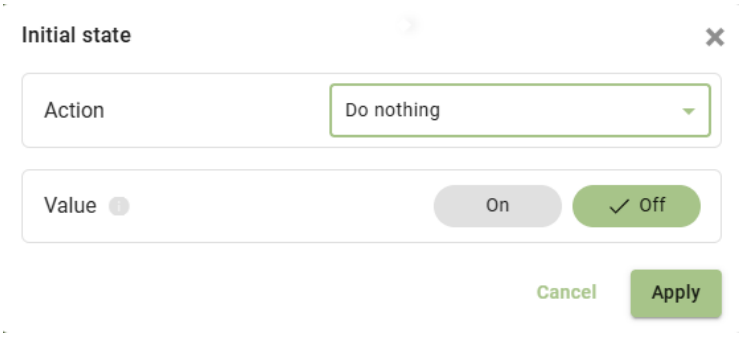
20.In that category, choose Single Switch:



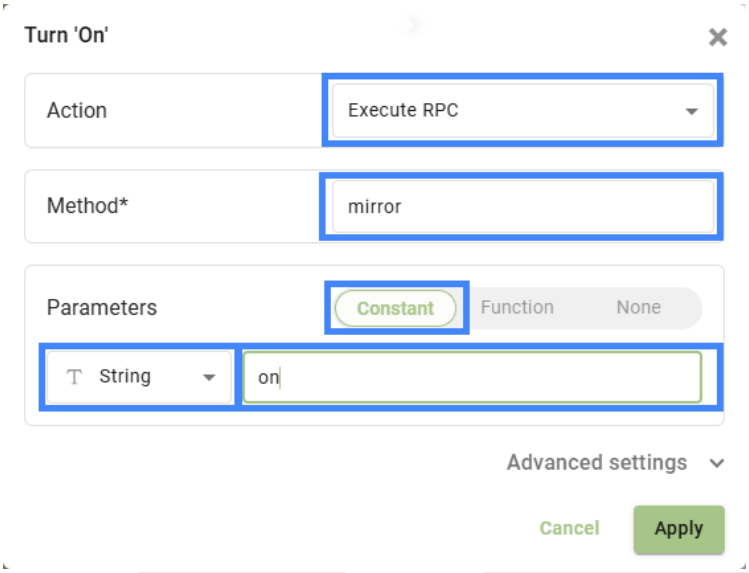
21. Choose your device based on the name assigned to you by your workshop host.



22. To setup Initial State, click on Execute RPC method 'getState':



23. To setup Turn ‘On’, click on Execute RPC method ‘setState(true)’:



24. To setup `Turn ‘Off’`, click on `Execute RPC method ‘setState(false)’` and set it up just as `Turn ‘On’`. Only replace the parameter value `on` with the value `off`.
25. Add the widget to the dashboard.
26. Save the dashboard.

You can now test the switch by toggling it at least three times. Observe the indicator created in chapter 5.1: Whenever the switch is turned `on`, the mirror-status switches to the value `off`. The only exception to this is the initial toggles, as the dashboard and the demonstrator are most likely out of sync.

Well done again!

Not only are you capable of visualizing data, you are now also able to interact with your device.

6 And now?

After this introduction, try to build a dashboard representing your demonstrator in a visually appealing way. Discover further functionality by playing around with the various widget classes available.

After getting familiar with dashboard creation, try to create a meaningful representation of the demonstrator you just connected. Expand your dashboard piece by piece. Maybe the following suggestions give you some inspiration:

- Visualize the device's heartbeat using a `value card`.
- Use a status indicator to display the mirror functionality introduced in chapter 5.
- Try adding multiple series to the same line chart.
- Try adding another control command to the software.

7 Some background information

Curious how the technology used in this guide works? This chapter gives you some background information about some of the key techniques.

7.1 About machine-to-machine communication

7.1.1 A mental model for communication

When talking about communication, there are two distinct dimensions to discuss:

- **Media** – describing **how** communication takes place.
- **Protocols** – describing **what** is communicated.

To ground this fairly abstract approach a bit, let's consider human communication: Two of the most common ways of humans communicate are through auditory or visual means. Each of these methods can be used by various languages such as English, Spanish, German, etc.

For a computer system it's very similar. Instead of auditive and visual media, the media available are WiFi, Ethernet, ZigBee, etc. while the language "spoken" are represented by protocols such as MQTT, HTTP/REST, etc. In the technical world, this coarse distinction is split even finer. However, we stay at this level of abstraction for the moment.

For further discussions however, it's worth keeping in mind, that any variant of these two dimensions can be combined virtually freely: MQTT can be sent through WiFi as well as through ZigBee, just as a Spanish sentence can be either spoken or written down.

7.1.2 The MQTT protocol

One of the de facto standards in the IoT world is the MQTT protocol. It defines the "language" in two different areas which are further described in below subsections.

7.1.2.1 How does a data transfer look like?

MQTT implements a so-called "publisher/subscriber model". The heart of the system is a MQTT broker (see also chapter 2), allowing sending devices to publish information, while other devices and services can subscribe to information published on that broker.

Whenever a device connects to the MQTT broker, it authenticates and then basically has the options to execute the following actions:

- publish new information (i.e. send information “upstream”)
- subscribe for specific information
- unsubscribe previously subscribed information
- disconnect

The connection might remain or be dropped until another transaction is necessary. If the connection persists, the device will from now receive on push notifications upon changing information.

If you feel like delving into the details around MQTT, consider looking at the following blog series covering MQTT essentials:

https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/

7.1.2.2 How can data be identified?

MQTT implements the very straightforward concept of a “key-value-store”. The keys, so-called “topics” hints how the information transferred has to be interpreted. Each topic is representing one piece of information. The payload is always user defined and the concrete format can be chosen by the developer.

Topic names can be virtually any string of characters (in theory even emojis), however for maximum compatibility it’s advised to only use the following character groups:

- `A-Z`, `a-z`
- `0-9`
- `_`, `-`

Some of the other characters have special meanings as for example the slash (`/`). This character is used to structure the topics in a tree, very similar to file paths in the filesystem on your computer. Other characters such as the double-cross (`#`) and others have also special meanings.

If you feel like delving into the details around topics in MQTT, consider looking at part 5 of HiveMQ’s MQTT essential series:

https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/

7.1.2.3 GBS/EduDemoS topic naming

For the EduDemoS adaption of ThingsBoard to work topic names on the MQTT-broker have to be structured as follows:

- EduDemoS
  - WorkshopA
    - SUNFLOWER01
    - SUNFLOWER02
  - WorkshopB
    - SUNFLOWER01
    - TURTLE01
    - TURTLE02

While the root node EduDemoS is necessary to separate EduDemoS related data from other data exchanged through the broker, the distinct workshop paths avoid interference of workshops running in parallel on the same system.

Above example illustrates the coexistence of two workshops, where WorkshopA has two sun-tracker demonstrators (SUNFLOWER01, SUNFLOWER02) running, while WorkshopB runs three demonstrators (SUNFLOWER01, TURTLE01, TURTLE02).

Regardless of what type of demonstrator is running, all devices use a specific prefix to distinguish between input and output topics:

- **input** topics are marked using the prefix **ctrl** underneath the device's node
- **output** topics are marked using the prefix **data** underneath the device's node

All of these rules result in a topic prefix for inputs as shown below:

```
EduDemoS/<Workshop ID>/<Demonstrator Type><Team ID>/ctrl
```

and for outputs (telemetry):

```
EduDemoS/<Workshop ID>/<Demonstrator Type><Team ID>/data
```

To make this a bit more concrete, let's have a look at the concrete topic name for the **mirror**-feature of team 01's SUNFLOWER attending WorkshopB. Applying the above structure and rules, the RPC topic name for the mirror-feature's input results in:

```
EduDemoS/WorkshopB/SUNFLOWER01/ctrl/mirror
```

While the mirror feature's output value is found in the following topic:

```
EduDemoS/WorkshopB/SUNFLOWER01/data/mirror
```

7.2 About ThingsBoard

ThingsBoard is an open-source platform, designed for managing IoT devices and collecting, processing and visualizing data. It therefor acts as a central hub where smart devices such as sensors (e.g. weather stations, trackers, etc.) can send their data using protocols such as MQTT, HTTP or CoAP. It makes it easy to store, monitor and analyse data in real time as well as controlling devices remotely. It not only features built-in dashboard functionality, it also provides a rule engine allowing for automating responses to certain conditions (such as triggering an alarm if a sensor reports high temperature, etc.).

While the following subsections offer a brief overview of the core concepts, an in-depth documentation on ThingsBoard is available here:

https://thingsboard.io/docs/pe/

7.2.1 Dashboards

A dashboard is a customizable, graphical user interface, visualizing various information available in the system. Named after dashboards in automobiles, they resemble the very same idea in the IoT world: Giving a quick overview of the system's state.

In pursuit of this goal, dashboards render information in various visually appealing variants. Depending on the needs the data is visualized as texts, gauges or even interactive diagrams. All of this usually in a no- to low-code[4] fashion.

7.2.2 Devices

Devices or so-called "things" are of course the heart of every Internet of Things-system. In ThingsBoard, devices represent physical objects interacting with the real world by collecting sensor data and/or reacting on commands. Each device must be registered in ThingsBoard to securely transmit its data using standard protocols.

The data stream received from the devices is called telemetry (see chapter 7.2.3) and is upon reception automatically assigned to the respective device in ThingsBoard, while data is sent towards the device using so-called remote procedure calls (RPCs) as described in chapter 7.2.4.

Each device in ThingsBoard is assigned a distinct name which is a combination of your workshop ID, device type and team number assigned.

[4] No-/Low-Code: A concept allowing to create applications without the use of program code. These environments typically use graphical representation of workflows, etc. to make them more accessible for non-programmers. Examples are: Blockly, Scratch, etc.

7.2.3 Telemetry

Telemetry is the data devices send towards ThingsBoard. ThingsBoard timestamps the data upon reception and stores it internally for later use in visualization or analysis. Telemetry data usually represents current sensor readings, as well as the inner status of the device allowing for in depth monitoring.

In the GBS/EduDemoS adaption of ThingsBoard, all telemetry received is expected to be sent through the MQTT-broker underneath the appropriate prefix for your workskhop (see chapter 7.1.2.3).

7.2.4 Remote Procedure Calls (RPC)

In order to interact with the device from a dashboard for example, ThingsBoard uses a mechanism called remote procedure calls or RPCs for short. RPCs allow dashboard builders to trigger action on the devices. Pretty much like function calls in programming languages, RPCs take arguments which are transmitted to the device.

The GBS/EduDemoS adaption of the RPC-system converts an RPC into an MQTT datapoint in the `ctrl` part of the target device (see chapter 7.1.2.3). The method name translates to the topic name, while the argument is used as value for the topic.

7.3 How the software solution works

To make the demonstrator talk to the MQTT broker, it is augmented with a communication part, which takes care of three things:

1. Connect and maintain the connection to the WiFi network configured.
2. Connect and maintain the connection to the MQTT broker.
3. Update the demonstrator's status after a certain period in order to save bandwidth and traffic volume.

For details on the software implementation of your demonstrator, please refer to the `README.md`-file in the respective directory and – of course – the source code.

8 Setting up your own workshop

In order to set up your own workshop including IoT uplink, or to reproduce the actions outlined in this guide, you'll need your own IoT infrastructure, consisting of:

- An MQTT broker (see chapter 7.1)
- An instance of ThingsBoard (see chapter7.2) or any other IoT management tool

For EduDemoS, the infrastructure can be hosted at and provided by the GBS St. Gallen, one of the partnering schools of the project. The infrastructure provided for EduDemoS is especially configured for the use cases outlined in this guide. There are also alternative providers for IoT solutions available in the market (e.g. Adafruit IO). However, for using these systems you have to adapt the software by yourself.

In case you want to conduct your own workshop including the IoT parts, feel free to register your own workshop using the following contact form (also scannable through QR code beside):

https://www.gbssg.ch/edudemos-event.html

For the best experience, please outline which demonstrators you intend to build during your workshop and how many instances of the demonstrators you intend to build (i.e. how many teams you will run). This will allow for optimal preparation of your experience.

Note: Make sure you have a 2.4 GHz WiFi network available at your workshop location. The WiFi network is expected to be configured as follows:

- SSID sent ("visible")
- It has to be setup with pre-shared key ("password")

Co-funded by the European Union

Gerda Stetter Stiftung
Technik *macht* Spaß!

EDUDEMOS

EDUcating through Sustainable **DEMO**nstrators

Get In Touch With Us

@edudemos

info@edudemos.eu @edudemos **www.edudemos.eu**

@edudemos

V1.0